

EXAM PLATFORM

# Internship Project Modules

## Volume II — Extended Modules

*Technical Specification & Assignment Guide*

### Web Development

5 Modules (M-11 to M-15)

### Artificial Intelligence

5 Modules (M-16 to M-20)

20 Interns · 2 per Module · 5–6 Weeks per Module · June 2026

# Module Overview — Volume II

---

| ID   | Domain  | Module Title                                          | Duration  | Interns |
|------|---------|-------------------------------------------------------|-----------|---------|
| M-11 | Web Dev | Live Exam Collaboration & Proctoring Control Room     | 5–6 Weeks | 2       |
| M-12 | Web Dev | Question Bank & Tagging System                        | 5–6 Weeks | 2       |
| M-13 | Web Dev | Multi-Tenant Organisation Management                  | 6 Weeks   | 2       |
| M-14 | Web Dev | Public API & Developer Portal                         | 5–6 Weeks | 2       |
| M-15 | Web Dev | Student Study Portal & Practice Mode                  | 5–6 Weeks | 2       |
| M-16 | AI      | AI-Powered Feedback & Hint Engine                     | 5–6 Weeks | 2       |
| M-17 | AI      | Natural Language Search & Intelligent Query Assistant | 5–6 Weeks | 2       |
| M-18 | AI      | Automated Exam Quality Assurance & Bias Detector      | 5–6 Weeks | 2       |
| M-19 | AI      | Intelligent Plagiarism & Academic Integrity Suite     | 5–6 Weeks | 2       |
| M-20 | AI      | Conversational AI Exam Chatbot & Virtual Invigilator  | 5–6 Weeks | 2       |

This is Volume II of the Exam Platform Internship Module Specifications, covering modules M-11 through M-20. Each module is designed for 2 interns working full-time over 5–6 weeks. All modules in this volume build upon and extend the platform introduced in Volume I (M-01 to M-10) and can be developed in parallel with those teams. Integration points with Volume I modules are noted within each specification.

# Live Exam Collaboration & Proctoring Control Room

*Real-time supervisor dashboard, live chat support, and incident management*

## Overview

During a live exam, supervisors have no centralized view of what is happening. This module builds a real-time Control Room dashboard where supervisors can watch all active exam sessions simultaneously, chat privately with individual students (for technical support), broadcast announcements mid-exam, extend time for specific students, pause or terminate sessions, and flag incidents. It also provides a live question-heat-map showing which questions are taking the most time across the cohort, enabling instructors to spot problematic questions in real time.

## Functional Requirements

- Live session grid: supervisor sees all active students as cards (name, progress %, time remaining, proctoring status, question currently on)
- Per-student controls: extend time, warn student, pause their session, terminate session, add private note
- Private student chat: supervisor opens a sidebar chat with any individual student; student sees it as a support widget
- Mid-exam broadcast: supervisor sends an announcement that appears as a dismissible banner on all student screens instantly
- Live question heat map: color-coded grid of all questions showing average time spent per question across active students (updated every 30 seconds)
- Incident log: supervisor flags an incident (type: suspected cheating, technical issue, etc.) with a note; linked to the student's session
- Auto-alerts: supervisor is automatically alerted when a student's proctoring system raises a flag (face absent, tab switch, etc.)
- Session analytics at a glance: count of submissions so far, average completion % across active students, estimated finish time

## Technical Architecture

- Real-time core: Socket.io rooms — one room per exam session; student clients emit heartbeat events with progress; supervisor joins the exam room as an observer
- State management: Redis stores live session state (student progress, current question, proctoring flags) with 5-second TTL heartbeat; supervisor dashboard reads from Redis via Node.js API
- Student chat: Socket.io private messaging using `socket.to(studentSocketId).emit`; messages persisted in DB for audit

- Broadcast: server emits to entire exam room; students receive and display banner; acknowledgement tracked
- Heat map computation: aggregated every 30 seconds by a BullMQ repeatable job reading time-on-question events from DB
- Frontend: React dashboard with CSS Grid layout for student cards; Recharts for heat map; Socket.io client for real-time updates
- Time extension: supervisor sends extension event; student client receives and updates countdown timer; backend validates and stores new end time

## Deliverables & Milestones

| Phase  | Deliverables                                                                                                      |
|--------|-------------------------------------------------------------------------------------------------------------------|
| Week 1 | Socket.io room architecture, student heartbeat protocol, Redis session state, basic supervisor grid UI            |
| Week 2 | Per-student controls (extend time, pause, terminate), incident log, auto-alert integration with proctoring module |
| Week 3 | Private student chat (send/receive, history), mid-exam broadcast with acknowledgement tracking                    |
| Week 4 | Live question heat map (aggregation job + visualization)                                                          |
| Week 5 | Session-level analytics widgets, supervisor notification sounds/badges for proctoring flags                       |
| Week 6 | Load testing (100+ concurrent students), mobile-responsive supervisor layout, documentation                       |

## Non-Functional Requirements

- Dashboard must reflect student state changes within 3 seconds
- Must support monitoring 200+ concurrent students without UI lag
- All chat messages and incident logs persisted and retrievable post-exam
- Supervisor actions (pause, terminate) must take effect on student screen within 2 seconds

## Question Bank & Tagging System

*Centralised multi-exam question repository with rich tagging, versioning, and review workflow*

### Overview

Currently, questions belong directly to a specific exam and cannot be reused. This module builds a centralized Question Bank — a shared repository of questions that can be tagged, searched, versioned, and reused across multiple exams. It includes a peer-review workflow (questions go through Draft → Review → Approved states before being usable in an exam), a tag taxonomy manager (topics, difficulty, Bloom's level, course), and a bulk-import pipeline. The question bank is the foundation for smarter exam assembly.

### Functional Requirements

- Centralised repository: all question types (MCQ, SQ, LQ, Code) stored in one bank, decoupled from individual exams
- Rich tagging: tag by topic (hierarchical, e.g. Programming > Python > Lists), difficulty, Bloom's level, course, language, and custom labels
- Tag taxonomy manager: admin creates/edits/merges/deletes tags; tag hierarchy with parent-child relationships
- Versioning: every edit to a question creates a new version; instructor can view full history and revert; exams pin to a specific version so live exams are unaffected by edits
- Review workflow: question moves through Draft → In Review → Approved → Archived states; reviewer can leave comments on specific fields
- Exam question picker: when building an exam, instructor picks questions from the bank (filters by tag, difficulty, type) and adds to sections; question is linked by reference, not copied
- Usage analytics: per-question stats (how many exams used it, average student score, discrimination index)
- Bulk import: import questions from XLSX template into the bank (with automatic Draft status)
- Duplicate detection: on import or creation, flag if a very similar question already exists in the bank

### Technical Architecture

- Database: questions table (id, content JSON, type, created\_by, status, current\_version\_id); question\_versions table (version number, content snapshot, changed\_by, changed\_at); question\_tags junction; tags table (id, name, parent\_id, slug)
- API: RESTful CRUD for questions, tags, versions; filter endpoint supporting multi-tag AND/OR queries
- Frontend: question bank browser (filterable, sortable table + card view toggle); question editor (existing); tag manager (tree view)

- Review workflow: status transitions enforced server-side with role checks (only reviewers can approve); email notifications on status change (via M-05 if available)
- Versioning: immutable version records; exam\_question\_links stores question\_id + version\_id; when exam is live, version is locked
- Search: PostgreSQL full-text search (tsvector) on question content + tag names; fallback to pgvector semantic search if M-09 is available

## Deliverables & Milestones

| Phase         | Deliverables                                                                                                       |
|---------------|--------------------------------------------------------------------------------------------------------------------|
| <b>Week 1</b> | Database schema (questions, versions, tags, workflow states), migration from current exam-linked questions to bank |
| <b>Week 2</b> | Tag taxonomy manager UI (tree view, CRUD, merge), tagging API                                                      |
| <b>Week 3</b> | Question versioning (create version on edit, history viewer, revert, exam version pinning)                         |
| <b>Week 4</b> | Review workflow (status transitions, reviewer comments, email notifications), role-based access                    |
| <b>Week 5</b> | Exam question picker integration, usage analytics per question, full-text + tag search                             |
| <b>Week 6</b> | Bulk import with duplicate detection, performance testing (10,000+ question bank), documentation                   |

## Non-Functional Requirements

- Question bank search must return results within 500 ms for a bank of 10,000 questions
- Version history must be immutable — no deletion of historical versions
- Live exam question content must be unaffected by any edits to the bank question after the exam starts
- Tag taxonomy must support at least 3 levels of hierarchy

# Multi-Tenant Organisation Management

*White-label support, tenant isolation, billing limits, and admin hierarchy*

## Overview

The platform currently operates as a single-tenant system for one organisation. This module refactors it into a multi-tenant architecture where different organisations (universities, companies, training providers) each get their own isolated workspace — their own users, exams, question banks, and branding — all on the same infrastructure. A super-admin panel manages tenant provisioning, plan limits (max exams, max students, max storage), and usage monitoring. Each tenant can white-label the platform with their logo, colours, and custom domain.

## Functional Requirements

- Tenant provisioning: super-admin creates a new tenant (organisation name, slug, plan, admin email); system bootstraps isolated workspace and sends invite to tenant admin
- Data isolation: every DB table gains a `tenant_id` column; all queries scoped to tenant; no cross-tenant data leakage
- Plan limits: per-tenant limits on max active exams, max enrolled students per exam, max storage (for exports/certificates), max API calls; enforced server-side with friendly error messages
- Usage dashboard: super-admin sees per-tenant usage (exams created, students enrolled, storage used, last active) on a management portal
- White-labelling: tenant admin uploads logo, sets primary colour, sets platform name; login page and UI headers reflect tenant branding
- Custom domain: tenant can map a custom subdomain (e.g. `exams.acme.com`) to their workspace; platform reads tenant from subdomain
- Tenant admin hierarchy: tenant has its own admin, instructors, and students — roles scoped within tenant
- Tenant suspension & deletion: super-admin can suspend (block login) or soft-delete a tenant; data retained for 30 days before purge

## Technical Architecture

- Row-level tenancy: add `tenant_id` to all tables; middleware reads tenant from JWT claim or subdomain and injects into all DB queries via a shared query helper
- Tenant resolution: middleware checks subdomain → looks up tenants table → sets `req.tenant`; falls back to default tenant for main domain
- Plan enforcement: a PlanGuard middleware reads tenant plan from cache (Redis) and increments usage counters; blocks operation if limit exceeded
- Branding: tenant config (logo URL, hex colour, name) stored in tenants table; served via `/api/tenant/config` endpoint; React app fetches on load and applies CSS variables

- Super-admin portal: separate React app (or protected route) listing all tenants with usage metrics, plan management, and suspension controls
- Database: tenants table (id, slug, name, plan, limits JSON, branding JSON, status, created\_at); all existing tables gain tenant\_id FK
- Migration: script to assign existing data to a default tenant without downtime

## Deliverables & Milestones

| Phase         | Deliverables                                                                                                  |
|---------------|---------------------------------------------------------------------------------------------------------------|
| <b>Week 1</b> | Database migration (add tenant_id to all tables, seed default tenant), tenant resolution middleware           |
| <b>Week 2</b> | Tenant provisioning API + super-admin portal (create, list, suspend tenants)                                  |
| <b>Week 3</b> | Plan limits enforcement (PlanGuard middleware, usage counters in Redis)                                       |
| <b>Week 4</b> | White-labelling (logo upload, colour config, branding API, React dynamic CSS variables)                       |
| <b>Week 5</b> | Custom subdomain routing, tenant admin hierarchy (scoped roles), usage dashboard for super-admin              |
| <b>Week 6</b> | Tenant suspension/deletion + 30-day purge job, penetration testing for cross-tenant data leaks, documentation |

## Non-Functional Requirements

- Zero cross-tenant data access must be verified by an automated test suite that queries as tenant A and asserts tenant B data is never returned
- Tenant resolution adds at most 5 ms overhead per request
- Branding must apply within one page load (no flash of default theme)
- Plan limit checks must be atomic to prevent race-condition overages under concurrent requests

# Public API & Developer Portal

*RESTful public API with API keys, rate limiting, webhooks, and interactive docs*

## Overview

Organisations want to integrate the exam platform into their own systems — LMS platforms, HR tools, custom dashboards. This module exposes a versioned public REST API (v1) with API key authentication, fine-grained scopes, rate limiting, and comprehensive webhook support so third-party systems can be notified of exam events in real time. A Developer Portal (a Next.js static site) serves interactive API documentation (Swagger UI), guides, and a self-service API key management page. A sandbox environment with seeded test data lets developers experiment without affecting production.

## Functional Requirements

- API key management: authenticated users generate API keys with a name, expiry, and selected scopes; keys can be revoked; usage stats shown per key
- Versioned API: all public endpoints under `/api/v1/`; breaking changes go to v2 with v1 sunset notice
- Core endpoints: exams (list, get, create, update), questions (list, create), enrollments (list, enroll, unenroll), results (list, get by student), certificates (get, verify)
- Rate limiting: per-API-key request limits (e.g. 1,000 req/hour); quota shown in response headers (`X-RateLimit-*`); 429 response with `Retry-After` on limit hit
- Webhooks: third parties register webhook URLs for events (`exam.started`, `exam.submitted`, `result.published`, `certificate.issued`); platform POSTs signed payloads; retry with exponential back-off on failure
- Sandbox environment: separate DB schema prefilled with sample data; sandbox API keys cannot affect production
- Developer portal: interactive Swagger UI, getting-started guides, code samples (cURL, Python, JavaScript), webhook event catalogue
- API request log: per-key log of last 1,000 requests (timestamp, endpoint, status, latency) viewable in the portal

## Technical Architecture

- API key auth: middleware extracts `Authorization: Bearer <key>` header, hashes key (SHA-256), looks up in `api_keys` table, checks scope for the route, attaches tenant + user context
- Rate limiting: Redis sliding window counter per API key; `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` headers on every response
- Webhooks: `webhook_subscriptions` table (`tenant_id`, `event_type`, `url`, `secret`); BullMQ worker sends POST with HMAC-SHA256 signature header (`X-Signature`); retries up to 5 times
- Developer portal: Next.js static site with Swagger UI (`swagger-ui-react`), MDX for guides; deployed separately (Vercel/Nginx)

- API request logging: middleware writes request metadata to api\_request\_logs table asynchronously (fire-and-forget); old logs purged after 30 days
- OpenAPI spec: auto-generated from route definitions (express-openapi-validator or tsoa); single source of truth for Swagger UI and validation

## Deliverables & Milestones

| Phase  | Deliverables                                                                                                  |
|--------|---------------------------------------------------------------------------------------------------------------|
| Week 1 | API key model (generate, hash, scope, revoke), auth middleware, rate limiting with Redis                      |
| Week 2 | Core API endpoints (exams, enrollments, results, certificates) with scope enforcement                         |
| Week 3 | Webhook system (subscriptions, BullMQ delivery, HMAC signing, retry logic)                                    |
| Week 4 | Sandbox environment (seeded schema, sandbox key flag), API request logging                                    |
| Week 5 | Developer portal (Swagger UI, getting-started guides, code samples, key management UI)                        |
| Week 6 | OpenAPI spec validation, SDK stub generation (openapi-generator), end-to-end integration tests, documentation |

## Non-Functional Requirements

- API response time p95 < 200 ms for read endpoints under normal load
- Webhook delivery must succeed within 30 seconds of the triggering event
- API keys must be stored as SHA-256 hashes only — plain-text key never stored after creation
- Public API must be fully described by the OpenAPI spec with no undocumented endpoints

## Student Study Portal & Practice Mode

*Self-paced practice exams, flashcards, performance tracker, and study planner*

### Overview

Beyond formal exams, students benefit from a self-paced study environment. This module builds a Student Study Portal where students can take practice versions of approved exams (without affecting official records), generate flashcards from question content, track their own performance trends over time, and use a smart study planner that schedules revision sessions based on their weak areas and the upcoming exam date. The portal is student-facing only and does not expose answer keys for live exams.

### Functional Requirements

- Practice mode: student can take any exam they are enrolled in as an unlimited practice run; answers are auto-graded (MCQ instant, SQ/LQ skipped or self-assessed); results are private to the student and not visible to instructors
- Practice history: list of all practice attempts with date, score, time taken, and section breakdown; trend chart over time
- Flashcard generator: from any question in a practice exam, student can add it to a personal flashcard deck; flashcard shows question on front, model answer on back; deck is per-exam
- Spaced repetition: flashcard review sessions use a simple SM-2 spaced repetition algorithm to schedule next review based on self-rated recall (Again / Hard / Good / Easy buttons)
- Weak-area radar: after 3+ practice attempts, system shows a radar/spider chart of performance by topic tag, highlighting the weakest areas
- Study planner: student enters the exam date; system generates a day-by-day revision schedule prioritising weak topics and including flashcard review sessions
- Leaderboard (optional, opt-in): practice score leaderboard visible to all enrolled students who opt in, for motivation
- Progress badges: gamification — earn badges for milestones (first practice attempt, 7-day streak, perfect MCQ score, etc.)

### Technical Architecture

- Practice attempt engine: reuses exam attempt UI but with a 'practice' flag; results stored in `practice_attempts` table separate from official results
- Flashcard store: flashcards table (`student_id`, `question_id`, `deck_id`, `front`, `back`, `sm2_interval`, `sm2_ef`, `next_review_at`)
- SM-2 algorithm: implemented server-side; after each review, interval and ease factor updated; client requests 'cards due today' sorted by `next_review_at`
- Weak-area computation: nightly BullMQ job aggregates practice attempt results by question tag for each student; stores in `student_topic_stats` materialized table
- Radar chart: Recharts RadarChart fed from `student_topic_stats` API endpoint

- Study planner: simple algorithm — days until exam divided across weak topics (weighted by score gap); output is an array of day objects with topic + flashcard count; stored as a user preference (JSON in DB), editable
- Badges: badge\_definitions table (name, icon, condition JSON); badge\_awards table; a post-practice-attempt hook evaluates conditions

## Deliverables & Milestones

| Phase  | Deliverables                                                                                   |
|--------|------------------------------------------------------------------------------------------------|
| Week 1 | Practice mode engine (attempt flow, auto-grade, private result storage), practice history page |
| Week 2 | Flashcard generator (add from question, deck management, front/back UI)                        |
| Week 3 | Spaced repetition review session (SM-2 algorithm, self-rating buttons, due-today queue)        |
| Week 4 | Weak-area radar chart (topic stats aggregation job + Recharts RadarChart)                      |
| Week 5 | Study planner (algorithm, day-by-day schedule UI, edit/reschedule), progress badges            |
| Week 6 | Opt-in leaderboard, streak tracking, performance optimisation, documentation                   |

## Non-Functional Requirements

- Practice attempts must not be visible to instructors or admins in any grading view
- SM-2 scheduling must be accurate to the day and recalculate correctly after each review
- Flashcard deck must support at least 500 cards per student per exam without performance degradation
- Study planner generation must complete within 1 second

# AI-Powered Feedback & Hint Engine

*Contextual, pedagogically-sound hints during practice and personalized post-exam feedback reports*

## Overview

Students often get stuck during practice sessions with no guidance. This module builds an AI Hint Engine for practice mode that delivers Socratic, scaffolded hints (not direct answers) when a student requests help on a question. Post-exam, the engine generates a personalised written feedback report for each student explaining why their answers were right or wrong, what concepts to review, and specific follow-up resources. All hints and feedback are generated by an LLM with a carefully engineered pedagogical prompt and are cached per question to control costs.

## Functional Requirements

- In-practice hint button: student clicks 'Get a hint' on any question during a practice session; system returns a Socratic hint (guides thinking without revealing the answer); up to 3 progressive hints per question (each more direct than the last)
- Hint caching: hints generated once per question (per difficulty level) and cached; student-specific context (their partial answer) may generate a personalised hint on top of the cached base
- Post-exam feedback report: after a practice attempt, student receives a written narrative report: overall summary, per-question analysis (what was correct/incorrect and why), concept gaps identified, and 3-5 recommended topics to study
- Instructor hint review: instructor can review and approve/override hints generated for questions in their exam bank; approved hints become canonical and are served directly (no LLM call)
- Feedback tone settings: admin configures the feedback tone per organisation (Encouraging / Neutral / Rigorous)
- Usage limits: free plan students get N hints per day; premium plan unlimited (enforced per tenant plan)
- Hint quality rating: after each hint, student rates it (thumbs up/down); low-rated hints flagged for instructor review

## Technical Architecture

- LLM: OpenAI GPT-4o with system prompt enforcing Socratic pedagogy rules: 'Do not reveal the answer. Ask a guiding question. Reference the relevant concept without solving it.'
- Hint levels: three prompt variants (Level 1: conceptual nudge, Level 2: partial direction, Level 3: near-direct); each cached separately
- Cache: hints stored in `question_hints` table (`question_id`, `level`, `hint_text`, `generated_at`, `approved_by`); served from DB on repeat requests — LLM only called on cache miss

- Post-exam report: a BullMQ job triggered after practice attempt completion; fetches attempt details + question answers, builds a structured prompt, calls GPT-4o, stores report as Markdown in practice\_reports table
- Instructor review UI: table of generated hints per exam with approve/edit/reject actions; approved hints bypass LLM on future requests
- Feedback PDF: optional — render the Markdown report as a PDF (pdfkit) for download

## Deliverables & Milestones

| Phase         | Deliverables                                                                                        |
|---------------|-----------------------------------------------------------------------------------------------------|
| <b>Week 1</b> | LLM prompt engineering (Socratic hint, 3 levels), hint API endpoint, caching layer                  |
| <b>Week 2</b> | In-practice hint button UI, progressive hint reveal (Level 1 → 2 → 3), hint rating (thumbs up/down) |
| <b>Week 3</b> | Post-exam feedback report generation (BullMQ job, GPT-4o structured prompt, Markdown storage)       |
| <b>Week 4</b> | Feedback report display UI (Markdown renderer, per-question accordion, concept gap list)            |
| <b>Week 5</b> | Instructor hint review panel (approve/edit/reject), tone configuration by tenant, usage limits      |
| <b>Week 6</b> | Feedback PDF export, cost tracking (tokens per report), accuracy benchmarking, documentation        |

## Non-Functional Requirements

- Hint must be returned within 5 seconds for cache miss (LLM call); under 100 ms for cache hit
- Hints must never directly state the correct answer — enforced by prompt constraints and validated by a secondary LLM guard check
- Post-exam report must be generated within 2 minutes of attempt completion
- LLM cost per feedback report must be estimated and logged; alert if cost exceeds configured threshold

# Natural Language Search & Intelligent Query Assistant

*Semantic platform-wide search, NL-to-filter query, and a conversational data assistant*

## Overview

The platform has multiple search boxes (questions, students, exams, results) each working independently with basic keyword matching. This module builds a unified semantic search layer across all entities — questions, students, exams, results, and certificates — using vector embeddings stored in PostgreSQL with pgvector. On top of this, a natural-language query interface lets admins ask questions like 'Show me all students who scored below 50% on the SQL exam last month' and the system translates the query into a structured database filter and returns results, no SQL knowledge required.

## Functional Requirements

- Unified search bar: a single search input (accessible from any page via keyboard shortcut) that searches across questions, exams, students, and results simultaneously, returning grouped results
- Semantic search: results ranked by semantic relevance (embedding cosine similarity) not just keyword match; handles synonyms, paraphrasing, and typos
- Faceted filters: after a search, filter by entity type, date range, exam, tag, score range with instant re-ranking
- Natural language query: admin types a question in plain English (e.g. 'students who failed Biology exam this semester'); system extracts intent, maps to DB filters, executes query, and shows results in a data table
- Query explainer: alongside NL query results, system shows the interpreted filters in human-readable form ('Showing results for: Exam = Biology, Score < 50%, Date range = Jan–Jun 2026') with edit controls
- Search analytics: log all search queries (anonymised); admin can view most common searches, zero-result searches, and click-through rates
- Saved searches: users can name and save a search query for quick re-access; NL queries can also be saved and re-run

## Technical Architecture

- Embeddings: text-embedding-3-small used to generate embeddings for all searchable entities; stored in entity\_embeddings table (entity\_type, entity\_id, embedding vector(1536)); pgvector IVFFlat index for fast ANN search
- Embedding pipeline: a BullMQ worker generates embeddings for new/updated entities; bulk backfill script for existing data
- Unified search API: /api/search?q=... endpoint; runs parallel pgvector queries for each entity type, merges and re-ranks results by score, returns grouped JSON

- NL-to-filter: GPT-4o with function calling; system prompt describes available filter fields and value types; model returns a structured filter JSON; backend validates and executes as a parameterized query
- Frontend: command-palette style search overlay (Cmd+K); faceted filter sidebar; NL query input with streaming response; result data table with export
- Search analytics: search\_logs table (query, result\_count, entity\_types, user\_id, timestamp); admin dashboard page using M-02 chart components

## Deliverables & Milestones

| Phase         | Deliverables                                                                                                           |
|---------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Week 1</b> | pgvector setup, embedding schema, backfill script for existing questions + exams, embedding pipeline worker            |
| <b>Week 2</b> | Unified search API (parallel entity search, merge, rank), command-palette UI                                           |
| <b>Week 3</b> | Semantic ranking tuning, faceted filters, search analytics logging                                                     |
| <b>Week 4</b> | NL-to-filter (GPT-4o function calling, filter JSON validation, query execution)                                        |
| <b>Week 5</b> | Query explainer UI, edit-filter controls, saved searches                                                               |
| <b>Week 6</b> | Search analytics dashboard, zero-result detection + suggestion, performance benchmarking (p95 < 300 ms), documentation |

## Non-Functional Requirements

- Unified search must return results within 300 ms p95 for a corpus of 10,000 questions
- NL query interpretation must complete within 3 seconds end-to-end (LLM + DB)
- NL queries must be parameterized — no raw SQL constructed from user input; SQL injection impossible by design
- Embedding index must be rebuilt without downtime when new entity types are added

# Automated Exam Quality Assurance & Bias Detector

*LLM-based review of exams for clarity, fairness, difficulty balance, and accessibility*

## Overview

Before an exam is published, it should be reviewed for quality. This module builds an automated Exam QA pipeline that uses an LLM to review every question in an exam for: clarity of wording, grammatical correctness, ambiguity in MCQ options, cultural or gender bias, accessibility (reading grade level), difficulty balance across the exam, and alignment to the stated learning objectives. The result is a QA report with a per-question scorecard and an overall exam health score, surfaced in the exam editor before publish.

## Functional Requirements

- QA trigger: instructor clicks 'Run QA Check' on any exam in Draft or Review state before publishing; QA runs asynchronously
- Per-question checks: clarity score (1–5), grammar issues list, MCQ-specific checks (is there a clearly correct answer, are distractors plausible, no 'all of the above'), ambiguity flag, bias flag (cultural, gender, socioeconomic), reading grade level (Flesch-Kincaid)
- Exam-level checks: difficulty distribution (Easy/Medium/Hard balance), topic coverage vs stated objectives, duplicate or near-duplicate questions, time estimate validation (estimated completion time vs allotted time)
- QA report: per-question table with issue severity (Info / Warning / Error); overall exam health score (0–100); actionable suggestion for each issue
- In-editor highlights: questions with QA issues are highlighted in the exam editor with a badge; clicking opens the QA detail panel for that question
- Auto-fix suggestions: for grammar issues and clarity problems, LLM suggests a reworded version; instructor can accept with one click
- QA history: track QA runs over time; show how the health score improved across runs
- Learning objective alignment: instructor inputs learning objectives as free text; LLM checks each question is relevant to at least one objective

## Technical Architecture

- LLM: GPT-4o with structured JSON output; one API call per question with a comprehensive review prompt; returns structured object (clarity\_score, grammar\_issues[], bias\_flag, bias\_reason, reading\_grade, mcq\_issues[], suggestion, rewrite)
- Batch processing: BullMQ job processes all questions in parallel (concurrency 5); results saved to qa\_results table as they complete
- Readability: Flesch-Kincaid computed locally (text-readability npm package or Python textstat) without LLM to save cost

- Exam-level aggregation: after all question jobs complete, a final aggregation job computes health score, difficulty distribution, duplicate detection (embedding similarity), and topic coverage
- Frontend: QA report page (sortable/filterable issue table, health score gauge, fix suggestion modal with accept/reject); exam editor badge overlay
- Database: qa\_runs (exam\_id, triggered\_by, status, health\_score, created\_at); qa\_question\_results (run\_id, question\_id, results JSON)

## Deliverables & Milestones

| Phase         | Deliverables                                                                                                 |
|---------------|--------------------------------------------------------------------------------------------------------------|
| <b>Week 1</b> | LLM prompt engineering (per-question review JSON schema), BullMQ batch job, QA results schema                |
| <b>Week 2</b> | Per-question clarity, grammar, MCQ-specific, and bias checks; readability score (local computation)          |
| <b>Week 3</b> | Exam-level checks (difficulty distribution, duplicate detection, time estimate), health score formula        |
| <b>Week 4</b> | QA report page (issue table, severity badges, health score gauge, QA history)                                |
| <b>Week 5</b> | In-editor question highlights with QA badges, auto-fix suggestion modal (accept/reject rewrite)              |
| <b>Week 6</b> | Learning objective alignment check, cost optimisation (cache results for unchanged questions), documentation |

## Non-Functional Requirements

- QA for a 50-question exam must complete within 3 minutes
- Bias detection must cover at minimum: gender pronouns, Western cultural assumptions, and socioeconomic references
- Auto-fix rewrites must not change the intended meaning of the question — validated by a secondary LLM check
- QA results for unchanged questions must be served from cache (no repeat LLM call) unless the question content has changed

# Intelligent Plagiarism & Academic Integrity Suite

*Cross-submission similarity, internet source matching, and integrity reporting*

## Overview

Academic integrity enforcement currently relies solely on the human reviewer. This module builds a comprehensive Plagiarism & Integrity Suite covering three axes: (1) cross-submission similarity — detecting when multiple students submitted very similar SQ/LQ/code answers within the same exam; (2) internet source matching — checking if a student's answer closely matches publicly available web content; and (3) AI-generated content detection — scoring the likelihood that an answer was written by an AI. All findings are compiled into a per-submission integrity report with an overall integrity score that instructors can review and act on.

## Functional Requirements

- Cross-submission similarity: after exam closes, compute pairwise semantic similarity between all SQ/LQ answers for the same question; flag pairs above a configurable threshold (default 85%)
- Code plagiarism: for code questions, use AST-based structural comparison (not just text diff) to detect copied code with variable renaming
- Internet source check: for SQ/LQ answers, search a curated set of sources (or use a plagiarism API such as Copyleaks or a self-hosted solution) and report the top matching URL with similarity %
- AI-content scoring: run each SQ/LQ answer through a classifier (local model or external API) that outputs an AI-generated probability score (0–100%)
- Per-submission integrity report: aggregated view of all three signals for one student's submission; overall integrity score; list of flagged issues with evidence
- Exam-level integrity dashboard: list of all submissions ranked by integrity concern; filter by signal type; bulk mark as 'reviewed'
- Instructor actions: mark a flagged submission as 'Confirmed Violation', 'False Positive', or 'Under Review'; actions logged with timestamp and note
- Threshold configuration: admin configures similarity thresholds, AI-content threshold, and which signals are active per exam

## Technical Architecture

- Cross-submission: generate sentence-transformer embeddings for all answers to a question; compute pairwise cosine similarity using NumPy batch operations (not one-by-one); store flagged pairs in `similarity_flags` table
- Code AST comparison: Python `ast` module (for Python code) or `acorn` (for JavaScript) to parse code into AST; compare AST structure with tree-edit-distance algorithm; language-specific parsers per supported language

- Internet check: integrate Copyleaks API or equivalent; queue each answer as an async BullMQ job to avoid blocking; store top match URL + similarity in results
- AI detection: use GPT-2 perplexity scoring (local, via transformers library) as a free signal; optionally integrate GPTZero API as a second signal; average both
- Backend: Python FastAPI service for ML-heavy tasks (embedding, AST comparison, perplexity scoring); Node.js main app triggers via HTTP after exam closes
- Frontend: integrity dashboard with sortable table (by integrity score), expandable row showing per-signal breakdown, side-by-side diff view for similar submission pairs

## Deliverables & Milestones

| Phase         | Deliverables                                                                                      |
|---------------|---------------------------------------------------------------------------------------------------|
| <b>Week 1</b> | Embedding-based cross-submission similarity (batch computation, pairwise flags), schema design    |
| <b>Week 2</b> | Code AST plagiarism detection (Python + JavaScript parsers, tree-edit-distance)                   |
| <b>Week 3</b> | AI-content scoring (GPT-2 perplexity local model, optional GPTZero API integration)               |
| <b>Week 4</b> | Internet source matching (Copyleaks API integration or self-hosted alternative, async job queue)  |
| <b>Week 5</b> | Per-submission integrity report, exam-level integrity dashboard, instructor action workflow       |
| <b>Week 6</b> | Threshold configuration UI, side-by-side diff view, false-positive rate evaluation, documentation |

## Non-Functional Requirements

- Cross-submission similarity for 500 answers to one question must complete within 5 minutes
- Code AST comparison must be language-aware — not fooled by simple renaming or whitespace changes
- AI detection score must be treated as a signal, never a verdict — UI must label it 'Possible AI Content' not 'AI Generated'
- All flags must be reviewable by a human before any action is taken — no automated grade changes

# Conversational AI Exam Chatbot & Virtual Invigilator

*RAG-powered student support chatbot, FAQ automation, and exam-rules enforcement assistant*

## Overview

Students and instructors frequently have questions about platform features, exam rules, and their own results that currently require manual support. This module builds a RAG (Retrieval-Augmented Generation) powered Conversational Chatbot embedded into the platform. Students can ask it about their results, upcoming exams, and study tips. During an exam, a restricted version acts as a Virtual Invigilator — it can only answer questions about exam rules and technical issues, never about question content. Instructors get a separate instance that can query analytics data and generate summary reports in plain English. All conversations are logged and an admin can review and improve the bot's knowledge base.

## Functional Requirements

- Student chatbot: answers questions about the student's own results, enrollment status, certificate availability, upcoming exam schedule, and general study advice; grounded in the student's actual platform data
- Virtual Invigilator (exam mode): a locked-down chatbot active only during an exam session; restricted knowledge base (exam rules, time remaining, how to flag a technical issue); refuses to discuss question content; escalates to supervisor (M-11) if needed
- Instructor chatbot: can query analytics ('What was the average score on last week's Python exam?'), generate summaries ('Give me a brief report on Section 3 performance'), and answer platform how-to questions
- RAG knowledge base: admin uploads documents (PDF or Markdown) to a knowledge base (e.g. exam rules, platform guide, FAQs); chatbot retrieves relevant chunks before generating answers
- Knowledge base manager: admin UI to upload, delete, and preview documents in the knowledge base; view which documents were cited in recent conversations
- Conversation history: all chatbot conversations stored per user; user can view past conversations; admin can review any conversation for quality assurance
- Escalation: chatbot detects when it cannot answer confidently (low retrieval score) and offers to escalate to a human supervisor (creates a support ticket or triggers M-11 supervisor chat)
- Feedback loop: user rates each chatbot response (thumbs up/down); low-rated responses flagged for admin review to improve the knowledge base

## Technical Architecture

- RAG pipeline: documents chunked (512 tokens, 50-token overlap) → embedded (text-embedding-3-small) → stored in PostgreSQL pgvector table (knowledge\_chunks); at query time, top-k chunks retrieved by cosine similarity and injected into LLM context
- LLM: GPT-4o with system prompt defining the bot's persona, allowed topics (enforced via role — student / invigilator / instructor), and citation instructions ('answer only based on retrieved context')
- Streaming: LLM response streamed to frontend via Server-Sent Events (SSE) for real-time typewriter effect
- Student data grounding: for student-specific questions, backend fetches relevant data (results, enrollments, exam schedule) and appends to the context alongside RAG chunks — data is fetched server-side, never exposed raw to the LLM prompt in a way the user can extract
- Knowledge base ingestion: PDF parsed with pdfplumber; Markdown parsed directly; chunking with LangChain TextSplitter or a custom implementation; embeddings generated and upserted via a background job
- Frontend: floating chat widget (React, collapsible); separate full-screen chat for instructor analytics chatbot; admin knowledge base manager page
- Conversation store: conversations table (user\_id, role, messages JSON array, created\_at); messages include role, content, retrieved\_chunk\_ids for citation tracking

## Deliverables & Milestones

| Phase         | Deliverables                                                                                                    |
|---------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Week 1</b> | RAG pipeline (chunking, embedding, pgvector storage, retrieval), GPT-4o integration, basic chat API             |
| <b>Week 2</b> | Student chatbot (data grounding for results/enrollment/schedule), chat widget UI with SSE streaming             |
| <b>Week 3</b> | Knowledge base manager (upload PDF/MD, chunk preview, delete, citation tracking)                                |
| <b>Week 4</b> | Virtual Invigilator mode (exam-locked system prompt, escalation to supervisor, refusal for off-topic queries)   |
| <b>Week 5</b> | Instructor analytics chatbot (NL query over analytics data, summary generation), conversation history UI        |
| <b>Week 6</b> | Feedback loop (rating, admin review), confidence-based escalation, hallucination rate evaluation, documentation |

## Non-Functional Requirements

- First token of chatbot response must appear within 1 second (streaming)
- Virtual Invigilator must refuse to discuss question content in 100% of test cases — evaluated by a red-teaming test suite
- RAG retrieval must not surface another student's personal data in any response — retrieval is scoped by tenant and (for personal data) by user
- Knowledge base ingestion of a 100-page PDF must complete within 5 minutes